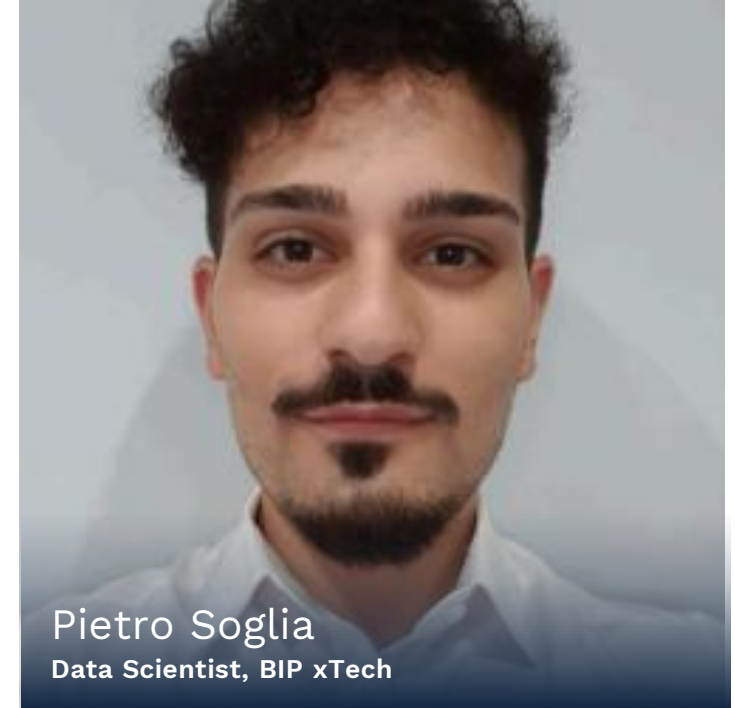


Machine Learning per l'analisi finanziaria





+39 348 233 8245
enrico.huber@bip-group.com



+39 347 267 8420
pietro.soglia@bip-group.com



Il churn come problema di classificazione: EDA



Dal dato grezzo al dataset modellabile

Modelli di classificazione e metriche di valutazione

Feature importance e interpretabilità

Modellazione avanzata e simulazione di progetto

Agenda del corso

MACHINE LEARNING PER L'ANALISI FINANZIARIA

Dal dato grezzo al dataset modellabile

Machine Learning per l'Analisi Finanziaria

Obiettivi della lezione

1	Rimuovere variabili non predittive e leakage identificate
2	Comprendere il problema dello sbilanciamento e selezionare metriche corrette
3	Conoscere e confrontare le principali strategie di gestione dell'imbalance
4	Identificare e trattare outlier con il metodo IQR
5	Creare feature ingegnerizzate motivate dall'EDA
6	Effettuare uno split train / validation / test
7	Costruire una Pipeline sklearn riproducibile
8	Salvare il dataset "modellabile" pronto per la Lezione 3

Rollout Plan



Cosa sappiamo sui nostri dati?

Aspetto	Valore / Osservazione
Shape	10000 righe x 18 colonne
Target (Exited)	Churn rate = 20,38%
Missing values	Nessuno – tutte le colonne completamente popolate
Duplicati	Nessuno
Leakage confermato	<i>Complain</i> – corr. 0996 con target
Colonne identificative	<i>RowNumber, CustomerID, Surname</i> – da escludere
Segnale forte	<i>Age</i> ($r = +0.285$), <i>IsActiveMember</i> ($r = -0.156$), <i>Balance</i> ($r = +0.119$)
Bimodalità <i>Balance</i>	36% dei clienti ha saldo zero; churn rate diverso: 14% (saldo=0) vs 24% (saldo>0)
Feature rumore	<i>EstimateSalary, Satisfaction Score, HasCrCard, Point Earned</i> - $r \approx 0$
Baseline	Logistic Regression: ROC-AUC ≈ 0.77 , recall churn = 21.1%

Il piano di oggi

Scoperta (EDA)	Azione (PREPROCESSING)
Complain — correlazione 0.996 con target	Rimuovere: leakage confermato
RowNumber, CustomerId, Surname — non predittivi	Rimuovere: identificativi
Bimodalità di Balance — churn rate diverso per saldo=0	Creare feature <i>balance_is_zero</i>
Churn rate 20.38% — accuracy fuorviante	Discutere sbilanciamento e metriche alternative
Feature numeriche su scale molto diverse (Balance 0–200k vs Tenure 0–10)	Feature scaling (<i>StandardScaler</i>)
Variabili categoriche: Geography, Gender, Card Type	Encoding (<i>OneHotEncoder</i>)

Ogni operazione di oggi ha una motivazione quantitativa, non è una scelta arbitraria

01

IL CHURN COME PROBLEMA DI CLASSIFICAZIONE



01

Pulizia del
dato

PULIZIA DEL DATO

Rimuovere il
rumore
prima di
modellare

L'introduzione di variabili
"rumore" nel modello non è solo
inutile: può degradare le
performance, aumentare la
varianza e — nel caso di variabili
leaky — produrre stime di
performance irrealisticamente
ottimistiche. La pulizia viene
prima di tutto.



RowNumber

E' un indice artificiale di riga, che
rappresenta un indicatore di
conteggio – non ha alcun significato
predittivo, in quanto l'ordinamento
è arbitrario



CustomerID

Identificativo univoco del cliente,
utilizzato eventualmente per join con
altre tabelle – non ha alcun valore
predittivo in quanto ogni nuovo
cliente avrà un customerID diverso
dal precedente



Surname

Il cognome non è una feature
comportamentale: un modello
addestrato su cognomi non può
generalizzare su clienti futuri con
cognomi diversi o nuovi.



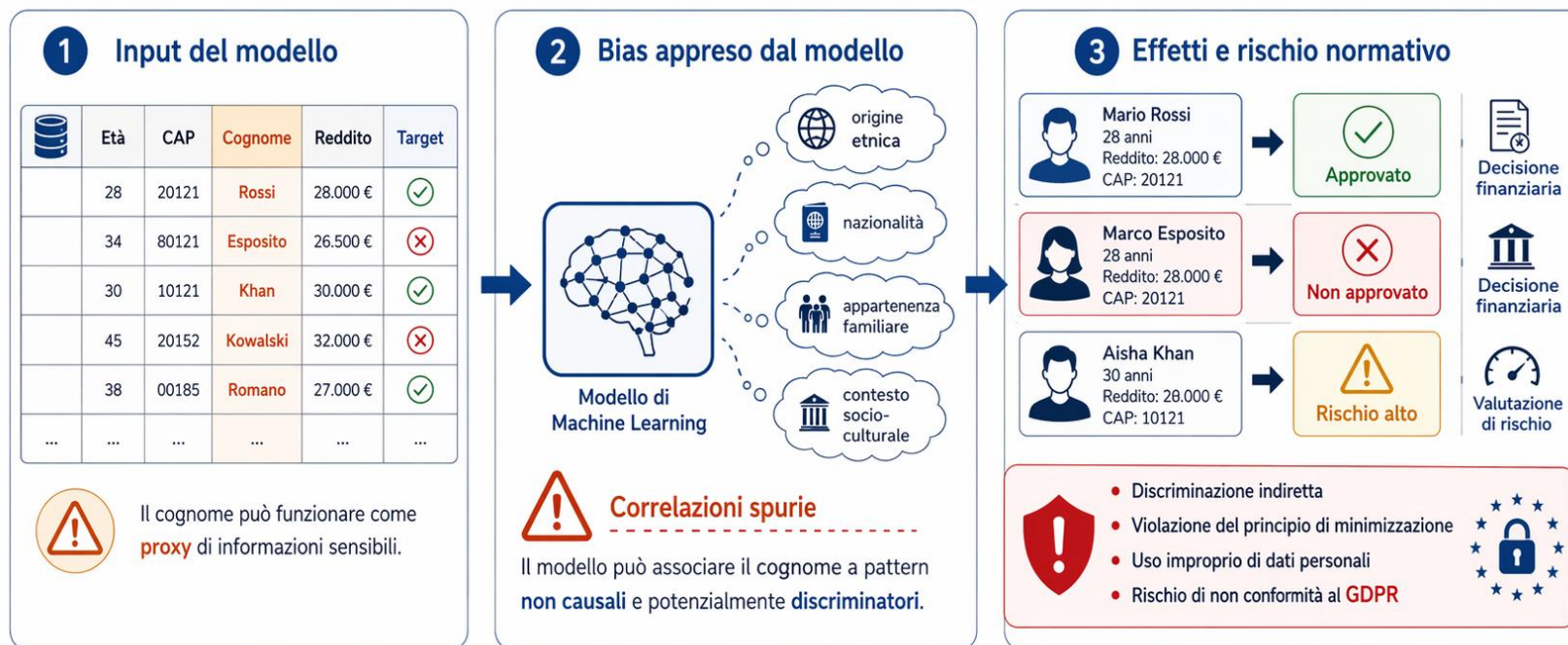
Complain

Si tratta di una variabile post-evento: il
reclamo viene registrato insieme o dopo la
decisione di churn, non prima. Includerla
nel modello produrrebbe performance
artificialmente altissime — ma il modello
sarebbe inutile in produzione, dove il
reclamo futuro non è ancora noto.

Focus sul campo Surname

Perché usare il "Cognome" come feature può essere pericoloso

Bias, proxy di attributi sensibili e implicazioni GDPR



Un modello non dovrebbe usare il cognome se non esiste una **base giuridica chiara** e una **reale necessità**.





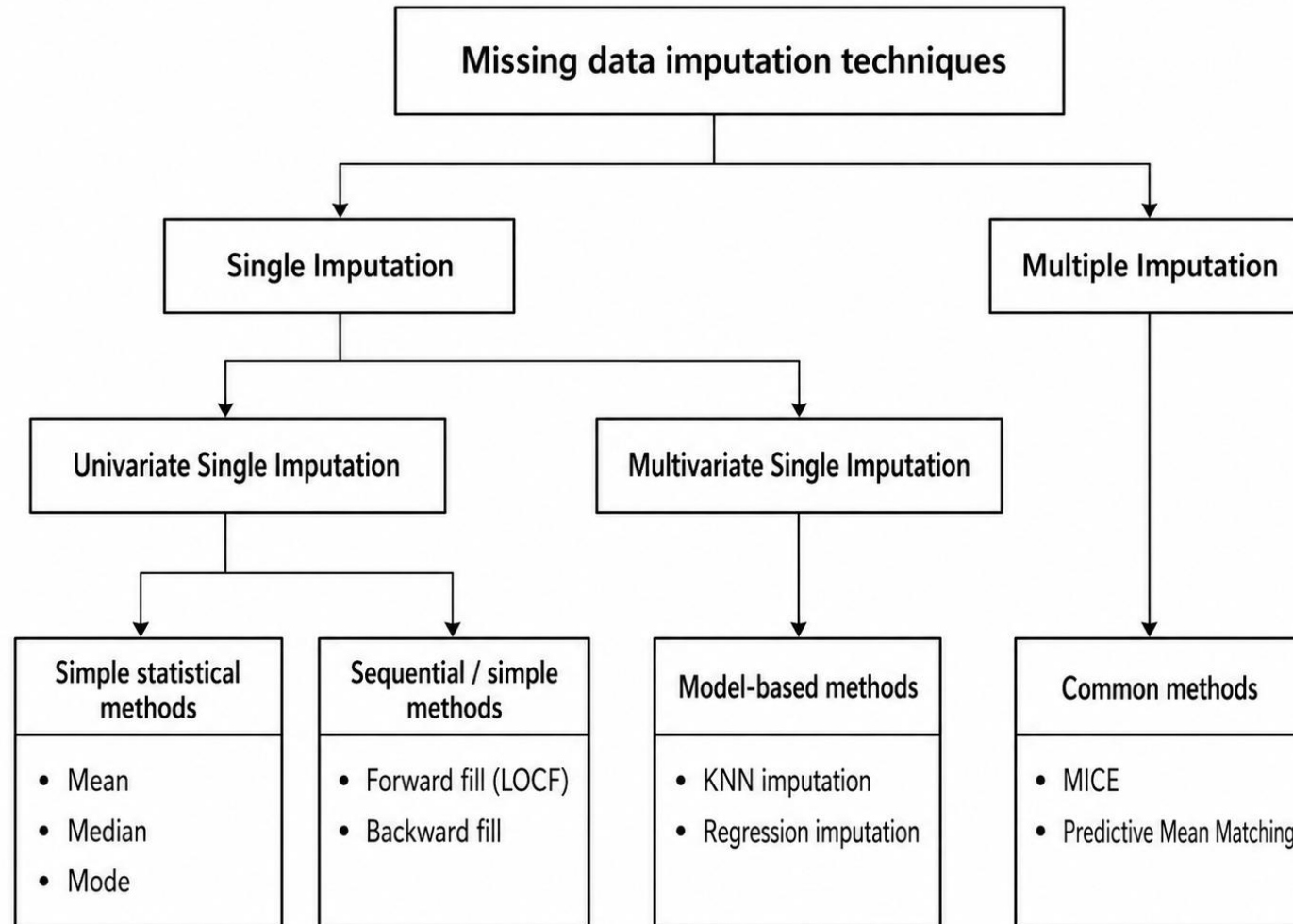
Rimuovere prima di modellare

```
Shape originale: (10000, 18)  
Shape dopo pulizia: (10000, 14)  
Colonne rimosse: ['RowNumber', 'CustomerId', 'Surname', 'Complain']
```



Il dataset passa da 18 a 14 colonne!

Cenni sulla missing data imputation



02

IL CHURN COME PROBLEMA DI CLASSIFICAZIONE



02

Gestione
Imbalance

Il paradosso dell'accuracy

Analizziamo come si comporta un DummyClassifier

- Un classificatore che predice sempre la stessa classe in output viene definito DummyClassifier
- Si tratta di un classificatore quasi sempre inutile, ma nel caso di classi sbilanciate può dare risultati anomali

79.62% accuracy, 0% recall

Performance di un classificatore che predice sempre $Y = 0$ (resta)

Analisi delle metriche per problemi di classificazione

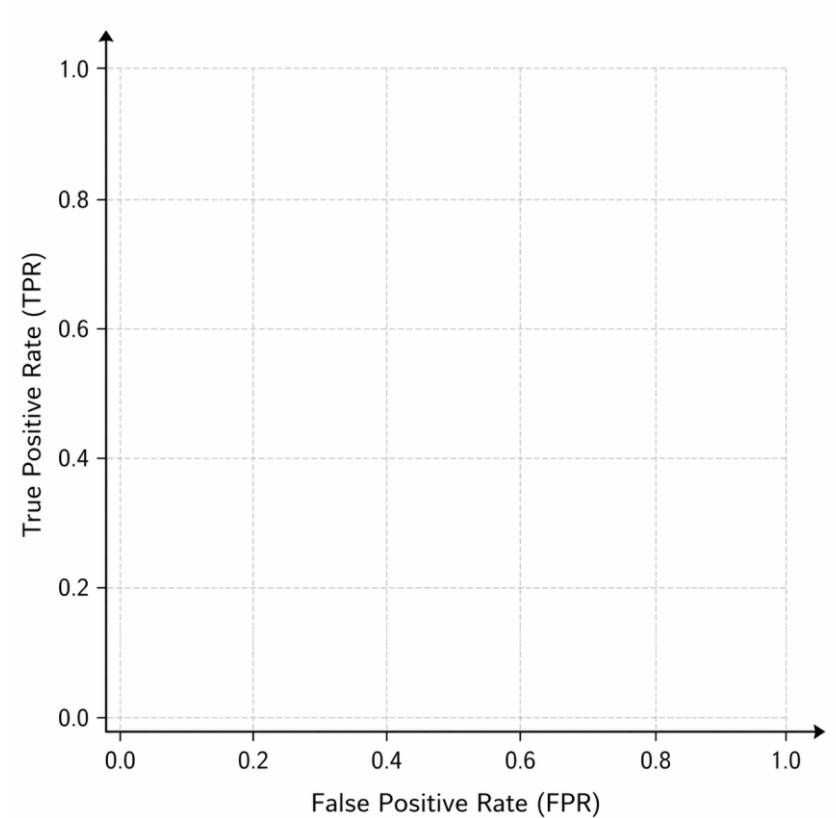
Precision, Recall, F1 score e altre

Metrica	Formula	Quando usarla
Accuracy	$\frac{t_p + t_n}{t_p + t_n + f_p + f_n}$	Metrica di default, non adatta per classi sbilanciate
Precision	$\frac{t_p}{t_p + f_p}$	Alta precisione → meno falsi positivi
Recall	$\frac{t_p}{t_p + f_n}$	Alta recall → meno falsi negativi
F1-score	$2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$	Media armonica di precision e recall

ROC-AUC Curve

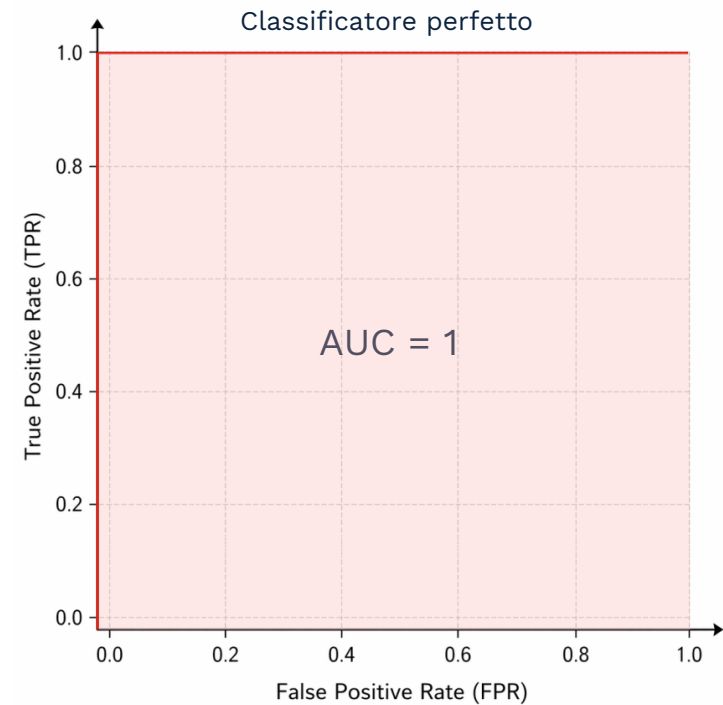
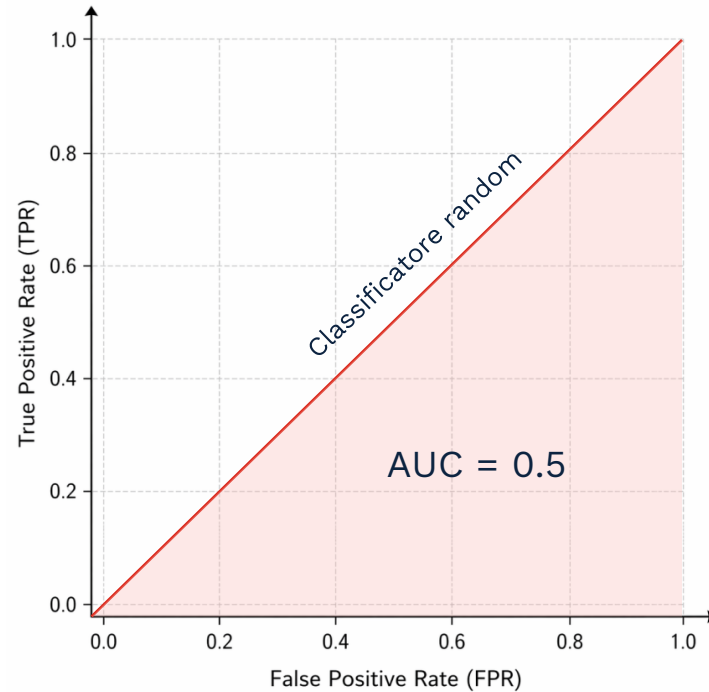
Receiver Operating Characteristic – Area Under the Curve

Curva che rappresenta
l'andamento del TPR (True
Positive Rate) in funzione del FPR
(False Positive Rate), **al variare
della soglia**

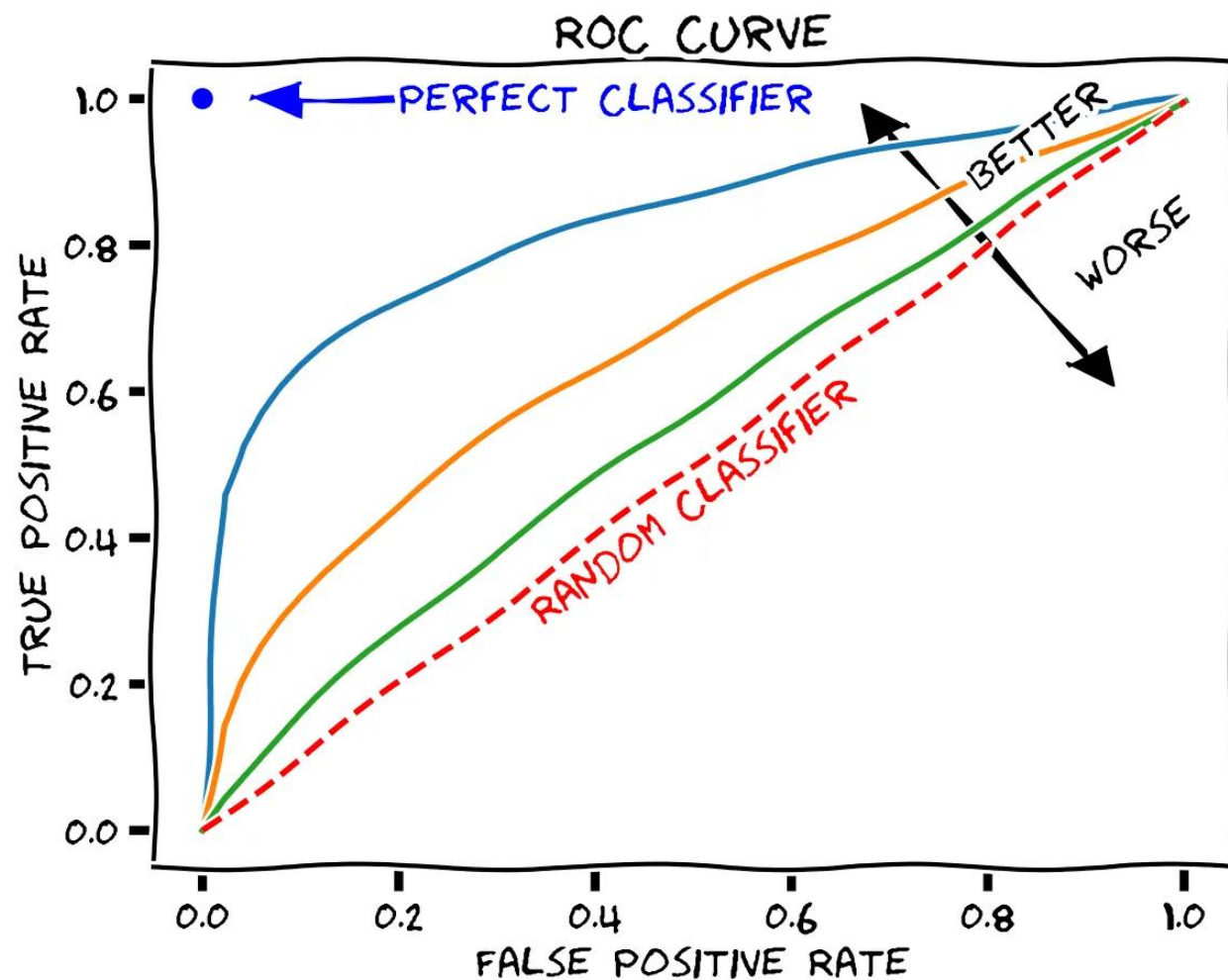


ROC-AUC Curve

Classificatore random e classificatore perfetto



ROC-AUC Curve



Matrice di confusione

		Predicted Values	
		Positive	Negative
Actual Values	Positive	TP	FN
	Negative	FP	TN

La scelta della soglia di classificazione sposta i valori nella matrice

Trade-off Precision - Recall

	Minimizzare FP (Precision ↑)	Minimizzare FN (Recall ↑)
Obiettivo	Contattare solo i veri churner – efficienza	Non perdere nessun churner – copertura
Quando	Budget retention limitato	Sistema di allerta precoce; alto valore del cliente
Rischio	Molti churner sfuggono senza essere contattati	Tanti contatti inutili; costo elevato

La soglia di default è 0.5, ma verrà scelta analizzando il Validation Set (Lezione 3)

Strategie per lo sbilanciamento

Strategia	Meccanismo	Pro	Contro	Quando usarla
Nessuna	Nessuna modifica	Semplice, baseline	Modello biased verso classe maggioritaria	Solo per capire il punto di partenza
<i>class_weight = 'balanced'</i>	Pesa inversamente la frequenza di classe nella loss	Nessun sample aggiuntivo; nessun rischio di leakage	Non tutti i modelli supportano <i>class_weight</i>	Prima scelta: semplice e stabile
Oversampling (SMOTE)	Crea campioni sintetici della classe minoritaria per interpolazione	Aumenta effettivamente il segnale della classe rara	Va applicato solo sui dati di training	Dataset piccoli o sbilanciamento severo (>1:10)
Undersampling	Rimuove campioni dalla classe maggioritaria	Riduce il tempo di training	Perde informazione; rischio underfitting	Dataset molto grandi (milioni di righe)
Threshold tuning	Abbassa la soglia di classificazione (es. 0.5 → 0.3)	Non modifica i dati; applicabile post-hoc	Richiede calibrazione; può degradare precision	Ottimizzare il trade-off dopo il training

class_weight e SMOTE

`class_weight = 'balanced'`

Modifica la funzione di perdita senza toccare i dati

Formula del peso per classe c :

$$w_c = \frac{n_{totale}}{n_{classi} \times n_c}$$

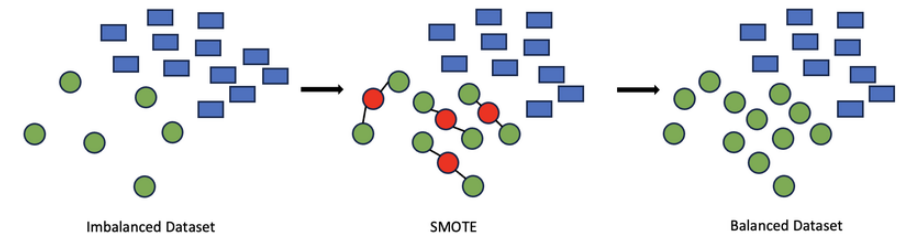
Risultato sul nostro dataset:

- Classe 0 (non churn, $n=7962$): peso = **0.628**
- Classe 1 (churn, $n=2037$): peso = **2.451**

→ Ogni errore su un churner vale **~2.45x** di più nella loss

SMOTE

crea campioni sintetici della classe minoritaria per interpolazione tra campioni reali vicini (k-nearest neighbors)



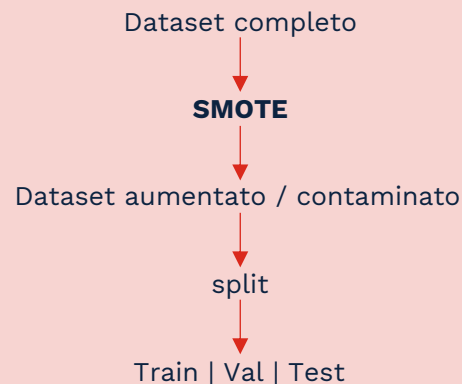
Risultato sul nostro dataset:

- Prima: 10000 campioni – churn: **20.38%**
- Dopo SMOTE: 15924 campioni – churn: **50%**

→ I campioni aggiuntivi non sono reali: sono **interpolazioni** nello spazio delle feature.

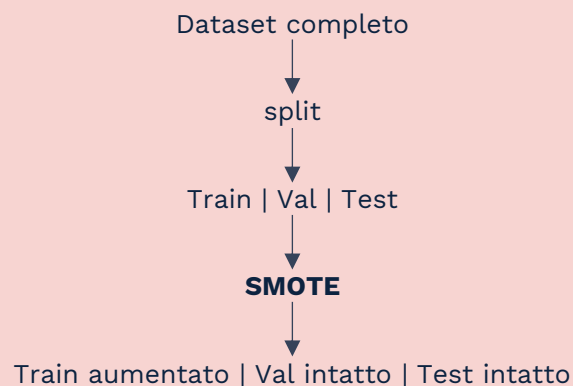
Regola aurea: SMOTE (e qualsiasi resampling) sempre **dopo** lo split

✗ SBAGLIATO – SMOTE prima dello split



I campioni sintetici nel test set derivano indirettamente dai dati di training. Performance sul test set **artificialmente ottimistiche**

✓ CORRETTO – SMOTE dopo lo split



Stima della performance sul test set **non contaminata**

03

Identificazione
Outlier



Metodo IQR per identificare gli outlier

$Q_1 = 25^{\circ} \text{ percentile}$

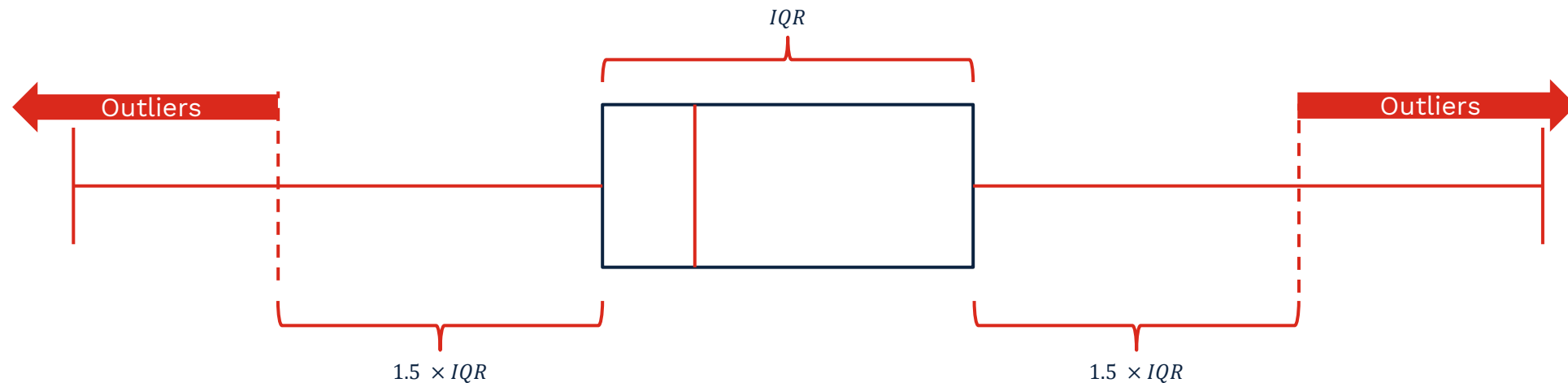
$Q_3 = 75^{\circ} \text{ percentile}$

$IQR = Q_3 - Q_1$

$Lower\ fence = Q_1 - 1.5 \times IQR$

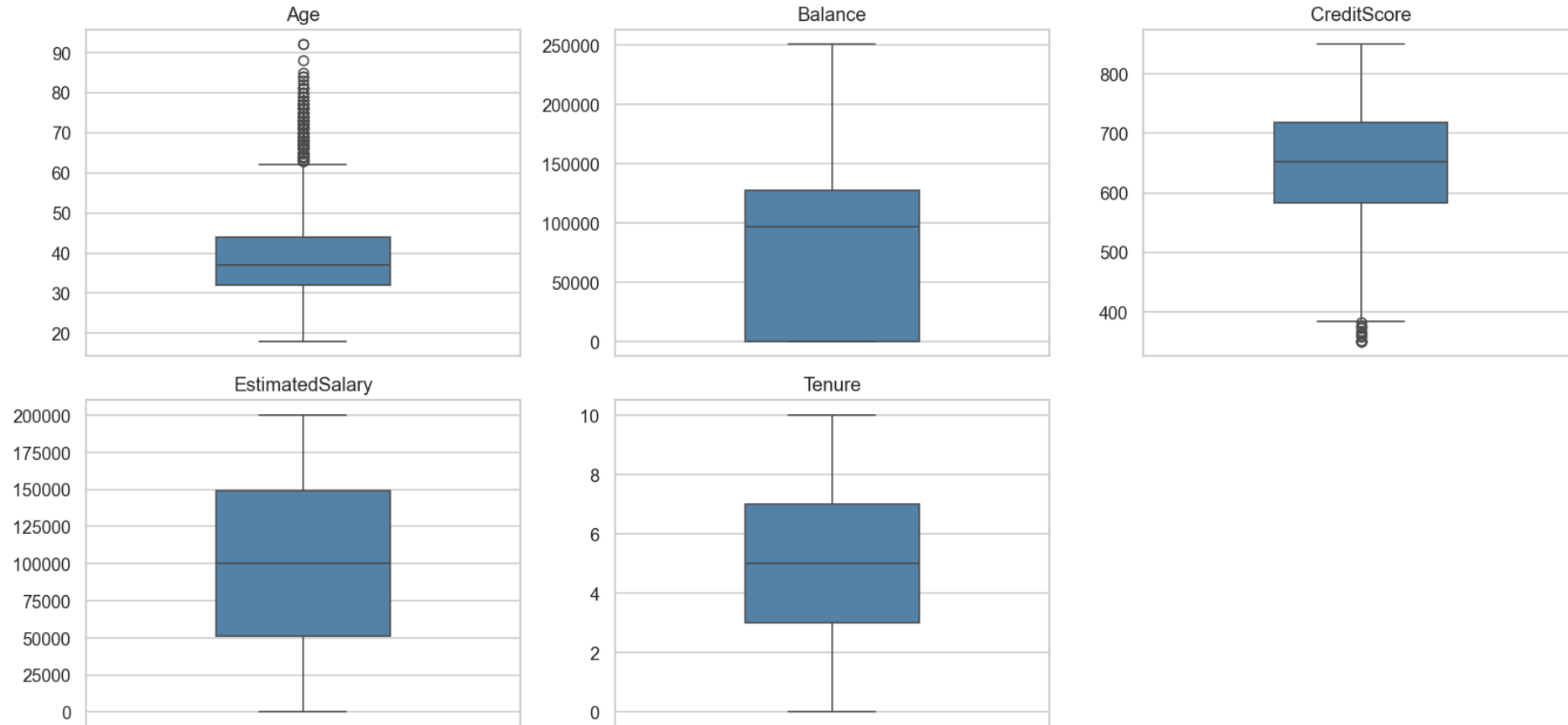
$Upper\ fence = Q_3 + 1.5 \times IQR$

Il moltiplicatore 1.5 è la convenzione standard (Metodo Tukey). Con una soglia a 3.0 si identificano gli outlier estremi



Risultati IQR sul dataset

Outlier per feature numeriche chiave (metodo IQR)



04

IL CHURN COME PROBLEMA DI CLASSIFICAZIONE

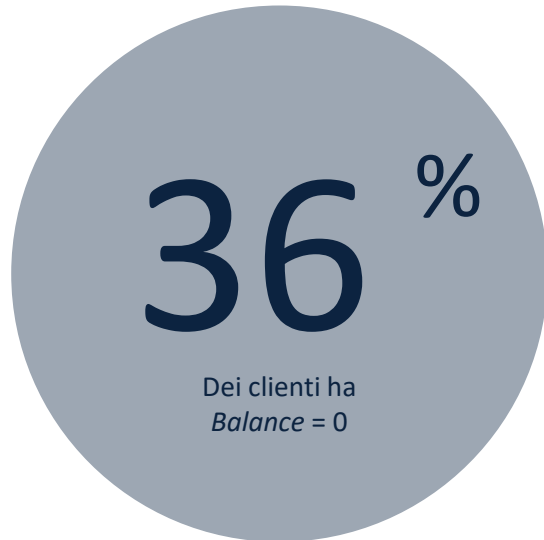


04

Feature
Engineering



Usare la EDA per modellare il fenomeno



Obiettivo della parte di feature engineering è catturare la bimodalità della feature *Balance*

La distribuzione di *Balance* è **bimodale**: picco a 0 + distribuzione continua intorno a ~100000

Un modello lineare tratta *Balance*=0 e *Balance*=1 come quasi identici → perde questo segnale

05

Split e
costruzione
pipeline

Perché tre split?



Split	Dimensione	Scopo	Quando usarlo
Train	60% - 6000 esempi	Addestrare il modello + fittare il preprocessore	Durante il <i>.fit()</i>
Validation	20% — 2,000 esempi	Scegliere iperparametri, confrontare modelli, ottimizzare soglia	Durante il tuning — può essere usato più volte
Test	20% — 2,000 esempi	Stima finale della performance su dati mai visti	Una sola volta, alla fine — mai durante il tuning

Il test set è "intoccabile" durante tutto lo sviluppo. Ogni volta che guardi i risultati sul test set per prendere una decisione, si introduce un bias implicito.



Encoding: OrdinalEncoder vs OneHotEncoder

Come codificare le variabili categoriche

Le variabili non numeriche hanno bisogno di un encoding per poter essere utilizzate dai modelli. Esistono vari tipi di encoding.

Encoder	Quando usarlo	Esempio	Risultato
<i>OrdinalEncoder</i>	Variabili ordinali con ordine naturale e significativo	Taglia (S < M < L < XL)	Sequenza numerica 0, 1, 2, 3...
<i>OneHotEncoding</i>	Variabili nominali senza ordine	<i>Geography</i> : France, Germany, Spain	Una colonna binaria per valore



Geography	France	Germany	Spain
France	1	0	0
Germany	0	1	0
Spain	0	0	1



Feature Scaling

Perché e come

Nel nostro dataset

- *Balance* ha range [0, 250898]
- *Tenure* ha range [0, 10]

→ Senza scaling, *Balance* dominerebbe i modelli lineari

Scaler	Formula	Risultato	Sensibile agli outlier	Quando usarlo
Standard Scaler	$z = \frac{x - \mu}{\sigma}$	Media≈0, std≈1	Si	Distribuzioni approssimativamente normali
RobustScaler	$z = \frac{x - Q_2}{Q_3 - Q_1}$	Centrato sulla mediana	No	Presenza di outlier moderati/severi
MinMaxScaler	$z = \frac{x - x_{min}}{x_{max} - x_{min}}$	Valori in [0,1]	Molto	Reti neurali, algoritmi che richiedono input in range fisso

PIPELINE *ColumnTransformer*



```
ColumnTransformer

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

# Pipeline per feature numeriche: imputazione (no NaN qui, ma best practice) +
scaling
num_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")), # robusto agli outlier
        ("scaler", StandardScaler()),
    ]
)

# Pipeline per feature categoriche: imputazione (best practice) + OneHotEncoding
cat_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        (
            "encoder",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False,
            ),
        ),
    ]
)

# ColumnTransformer: combina le due pipeline
# remainder="drop" rimuove colonne non specificate
preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_pipeline, num_cols),
        ("cat", cat_pipeline, cat_cols),
    ],
    remainder="drop",
    verbose_feature_names_out=True,
)

# Fit su X_train SOLTANTO
preprocessor.fit(X_train)

# Transform su train, val e test
X_train_proc = preprocessor.transform(X_train)
X_val_proc = preprocessor.transform(X_val)
X_test_proc = preprocessor.transform(X_test)
```


06

IL CHURN COME PROBLEMA DI CLASSIFICAZIONE



06

Live
coding

Tieniti in contatto con il team



Enrico Huber

Data Scientist

+39 348 233 8245
enrico.huber@bip-group.com



Pietro Soglia

Data Scientist

+39 347 267 8420
pietro.soglia@bip-group.com



Grazie dell'attenzione